

Dear interviewers:

Thank you for taking the time to review my report on question 2, “**Particle Flow Filter and Differentiable Particle Filter Part 2: Stochastic Particle Flow and Differentiable PF**”. You can refer to more details in my [github repo](#).

Yours sincerely,
Lixin Tu

Particle Flow Filter and Differentiable Particle Filter

1 Part 1: From classical filters to particle flows

Please refer to our [previous submission](#).

2 Part 2: Stochastic Particle Flow and Differentiable PF

Introduce and evaluate stochastic particle flows for stability and design a differentiable PF pipeline with OT resampling.

Comment 2.1

- 1) Stochastic particle flow:
 - a) Other than the deterministic particle flow, we can construct stochastic particle flow as shown in Dai(21) and Dai(22). Please replicate the main results of Dai(22)

Response:

Particle filters often suffer from particle degeneracy in high-dimensional spaces. Particle flow algorithms mitigate this by migrating particles continuously from the prior to the posterior distribution via a stochastic differential equation (SDE). However, the resulting SDE can be numerically "stiff," leading to integration difficulties. To solve this problem, Dai et al. [Daum et al., 2017] proposed the stochastic particle flow to investigate the optimal log-homotopy strategy to mitigate such stiffness.

The core methodology involves defining a generalized linear log-homotopy and optimizing the flow schedule $\beta(\lambda)$. The intermediate density $p(x, \lambda)$ linking the prior $p_0(x)$ and the posterior $p_1(x)$ is defined as:

$$\log p(x, \lambda) = \alpha(\lambda) \log p_0(x) + \beta(\lambda) \log p_1(x) - \log \Gamma(\lambda) \quad (1)$$

where $\lambda \in [0, 1]$ is the pseudo-time, and $\Gamma(\lambda)$ is a normalization factor. A constraint is imposed such that $\alpha(\lambda) + \beta(\lambda) = 1$.

The particle dynamics are governed by the following diffusion process:

$$dx = f(x, \lambda)d\lambda + \sqrt{Q}dw_\lambda \quad (2)$$

where the drift $f(x, \lambda)$ is derived to satisfy the Fokker-Planck equation corresponding to the target density $p(x, \lambda)$. For Gaussian assumptions, the drift is given by:

$$f(x, \lambda) = K_1 \nabla_x \log p(x, \lambda) + K_2 \nabla_x \log h(x) \quad (3)$$

Here, the coefficients K_1 and K_2 depend on the Hessian of the density, $S = \nabla_x \nabla_x^T \log p$, and the diffusion matrix Q .

The "stiffness" of the SDE is closely related to the condition number of the Hessian matrix $M(\lambda) = -\nabla_x \nabla_x^T \log p$. The standard straight-line homotopy ($\beta(\lambda) = \lambda$) does not optimize this condition. To mitigate stiffness, an optimal control problem is formulated to find the schedule $\beta^*(\lambda)$ that minimizes the weighted sum of "energy" (rate of change of β) and the condition number $\kappa(M)$:

$$J(\beta, u) = \int_0^1 \left[\frac{1}{2} u^2 + \mu \kappa_\nu(M) \right] d\lambda \quad (4)$$

where $u = \dot{\beta}$ and μ is a weighting factor. The optimal β^* is obtained by solving the Euler-Lagrange equation, resulting in a Two-Point Boundary Value Problem (TPBVP):

$$\frac{d^2 \beta^*}{d\lambda^2} = \mu \frac{\partial \kappa_\nu(M)}{\partial \beta} \Big|_{\beta=\beta^*} \quad (5)$$

subject to boundary conditions $\beta^*(0) = 0$ and $\beta^*(1) = 1$.

To validate the method, we reproduce **Example 1** from the Dai(22) [Daum et al., 2017]. A stationary target at $x_{truth} = [4, 4]^T$ is observed by two passive infrared sensors located at $[3.5, 0]^T$ and $[-3.5, 0]^T$. Each sensor provides a bearing-only measurement z_i with noise variance $R = 0.04I_2$.

$$z = \left[\arctan\left(\frac{y_t}{x_t - 3.5}\right) \quad \arctan\left(\frac{y_t}{x_t + 3.5}\right) \right] + v \quad (6)$$

Herein, a Gaussian prior is assumed with mean $\mu_0 = [3.0, 5.0]^T$ and covariance $P_0 = \text{diag}(1000, 2.0)$. Same with the paper in Dai(22) [Daum et al., 2017], we use $\mu = 0.2$ for the optimization weight and a diffusion matrix $Q = \text{diag}(4.0, 0.4)$.

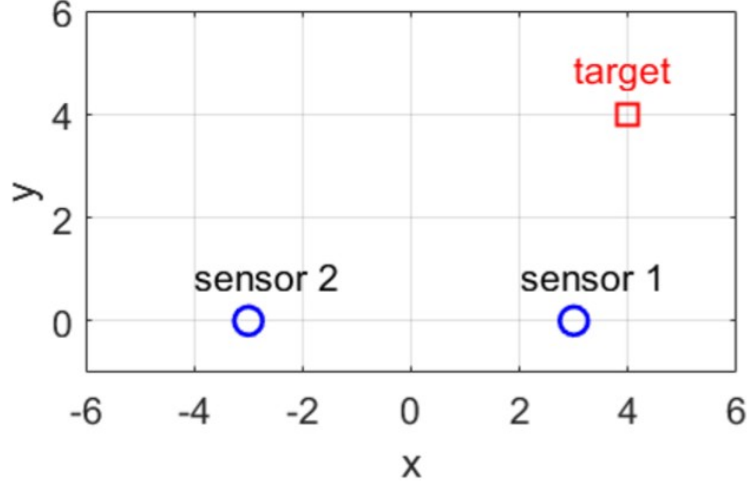


Figure 1: Scenario for Example 1

The scenario is graphically described in Fig. 1. We implemented the TPBVP solver to derive $\beta^*(\lambda)$ and simulated the SDE for $N = 50$ particles. Fig. 2 shows the comparison of the optimal $\beta^*(\lambda)$ with the straight-line log-homotopy $\beta(\lambda) = \lambda$. Figure 2(d) illustrates the reduction in the stiffness of the Jacobian matrix F . Note that R_{stiff} is plotted on a logarithmic scale to aid visualization. The reduction in the stiffness ratio is modest for this example. However, the impact on reducing estimation errors is significant, as shown in Table 1.

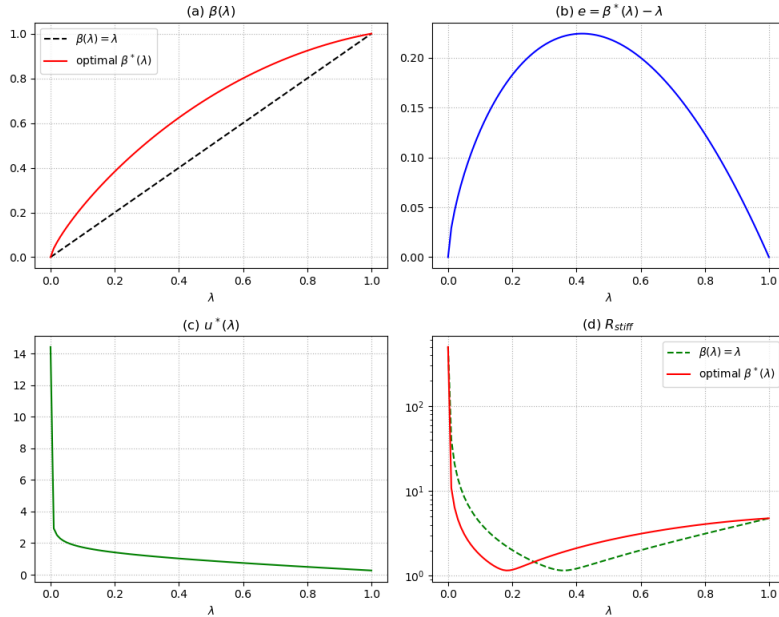


Figure 2: Replicated results for Example 1 comparing the optimal $\beta^*(\lambda)$ with $\beta(\lambda) = \lambda$: (a) $\beta(\lambda)$, (b) $e = \beta^*(\lambda) - \lambda$, (c) $u^*(\lambda)$, and (d) R_{stiff} .

We performed 20 Monte Carlo (MC) runs to evaluate filtering accuracy for both the baseline and optimal homotopies.

The results demonstrate that the optimal β^* achieves lower estimation error, as measured by RMSE_{β^*} , compared with the straight-line homotopy β_ℓ , which is consistent with the conclusions reported in the original paper. The optimal β^* induces a high initial drift velocity ($u > 10$), resulting in a stiff dynamical system; when standard integration with $N = 50$ steps is used, the numerical solution becomes unstable and the RMSE rapidly increases. To accurately capture the high-stiffness dynamics, this replication employs $N = 5000$ integration steps. Exact numerical agreement with the reported results is not expected due to differences in random seed initialization and the inherent sensitivity of the underlying stochastic differential equation.

Table 1: Performance Comparison for Example 1

MC Index	RMSE_{β_ℓ}	RMSE_{β^*}	$\text{tr}(P_{\beta^*})$	$\text{tr}(P_{\beta_\ell})$
1	8.7449	6.9176	9313.17	2719.83
2	6.6912	5.1138	8521.14	2951.22
3	6.6893	7.4003	7869.05	2803.14
4	6.9920	5.8582	6209.71	2173.71
5	7.8330	5.2330	7309.61	2438.19
6	16.0470	34.9286	26590.79	4658.42
7	33.2576	41.1393	15570.28	3960.87
8	41.7627	50.7629	7379.54	2107.09
9	35.1469	45.2230	5070.88	2265.37
10	32.4956	37.5864	3535.57	1684.15
11	12.1226	17.9549	15571.90	2765.76
12	5.6908	6.8697	7869.55	2827.91
13	12.0848	18.9043	7904.64	2013.00
14	11.2182	24.2460	10569.98	2398.15
15	6.1009	7.5875	9516.09	3289.95
16	6.6878	5.1840	8748.81	3036.75
17	30.0130	44.7369	16880.78	2551.41
18	24.8796	25.9677	2729.00	2040.15
19	28.6188	38.0955	8458.42	3098.98
20	42.0315	55.2316	19236.02	3153.52
Average	18.7554	24.2471	10242.75	2746.88

Fig. 3 compares the computational time of filtering with the determination of the optimal $\beta^*(\lambda)$ and that obtained by setting $\beta(\lambda) = \lambda$. The average time required to complete one Monte Carlo run is 4.70 s when the optimal $\beta^*(\lambda)$ is computed, compared with 3.65 s when this step is omitted, indicating that approximately 1.05 s of additional computation time is required to solve the associated two-point boundary value problem.

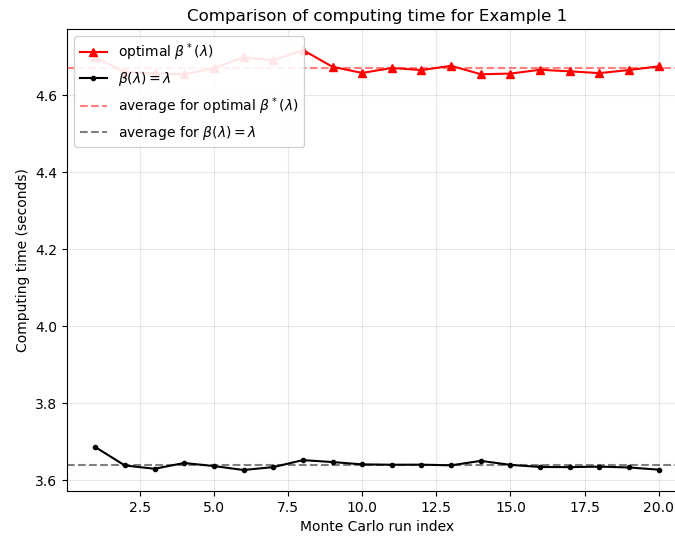


Figure 3: Replicated results for Comparison of computing time for Example 1.

Comment 2.2

b) Repeating part 1, if we use the optimized particle flow of Dai(22) as the proposal for the particle flow particle filter of Li(17), does it improve the result of the LEDH particle flow particle filter of Li(17)

Response:

We investigated and replicated the main results in Li(17) in Part 1 question 1.7. Herein, we replaced the particle flow particle filter (PFPF) framework with the optimized particle flow replicated in question 2.1. To compare the results, we used the same multi-target acoustic tracking simulation setup in question 1.7. The compared results are shown in Fig. 5.

Based on the five trials, the baseline PFPF_LEDH achieved a mean OMAT error of 0.6956, while PFPF_LEDH_Optimal achieved a lower mean error of 0.6530, indicating that using the optimized particle flow schedule from Dai (22) as the proposal for PFPF (Li (17)) improves tracking accuracy, although it suffers a higher computational cost. It is also found that the Boundary Value Problem solver encountered some stability issues and fell back to a linear schedule in some cases, indicating that improving the robustness of the BVP solver is a potential next step.

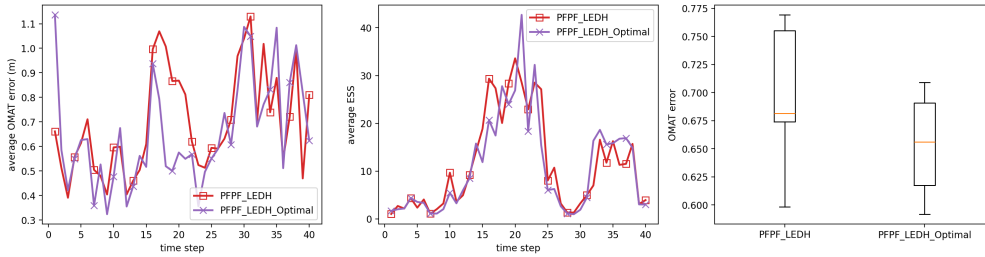


Figure 4: Replicated results for Comparison between PFPF_LEDH and LEDH_optimal .

Comment 2.3

2) Differentiable particle filter with OT resampling

i. Implement a differentiable PF following Corenflos(21) and Chen(23).

a) Start with soft-resampling (mixture with uniform) to enable backpropagation through weights

Response:

As shown in Corenflos (21) [Corenflos et al., 2021], the authors resampled from a mixture distribution rather than solely based on importance weights w_t . This is called the “soft-resampling”.

$$\tilde{w}_t^i = \alpha w_t^i + (1 - \alpha) \frac{1}{N} \quad (7)$$

where $\alpha \in [0, 1]$. This mixture strategy enables backpropagation through the importance weights. Since the weights $\tilde{w}_t^i$ are a differentiable function of the model parameters, TensorFlow can compute gradients with respect to θ for the weight calculation step. However, as illustrated in Figure 5, this approach introduces a significant gradient bias. The figure compares the gradient estimates under different stiffness parameters α :

- When $\alpha \approx 0$ (Uniform), the gradient is centered around -30. In this case, the resampling effectively ignores the observation likelihoods, decoupling the particle trajectory from the data quality.
- When $\alpha \approx 1$ (Standard PF), the gradient shifts significantly to approximately -65.

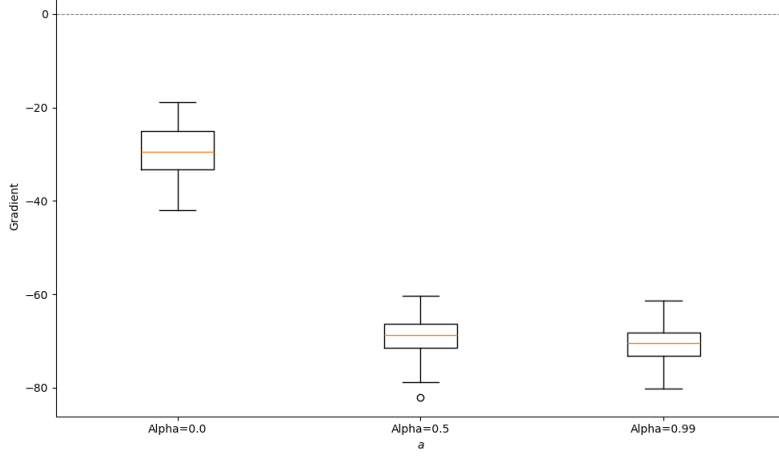


Figure 5: Gradient Analysis with soft-resampling.

While Soft Resampling allows gradients to flow through the computation of w_t , it still relies on a non-differentiable categorical sampling step to select particle indices. Consequently, the gradient estimator ignores the dependency of the post-resampling particle distribution on the parameters, leading to the biased estimates observed in the plot. This limitation necessitates the use of Optimal Transport resampling (OT) to ensure fully consistent gradient estimates.

Comment 2.4

b) Upgrade to entropy-regularized optimal transport (OT) resampling with Sinkhorn algorithm. Tune the regularization parameter and iterations for bias-variance-speed trade-off.

Response:

We implemented the Sinkhorn algorithm shown below [Corenflos et al., 2021] to compute the optimal transport matrix P_ϵ . This replaces the non-differentiable multinomial sampling with a differentiable matrix multiplication: $X_{new} = NP_\epsilon X_{old}$. We also analyzed the impact of the regularization parameter ϵ and the number of Sinkhorn iterations on the gradient estimate. As shown in Fig. 6, the small regularisation parameter ϵ (e.g., 0.1) leads to high variance and instability. With weak regularisation, the Sinkhorn iterations struggle to converge to the "hard" transport map in a finite number of steps, leading to numerical stiffness. Increasing ϵ smooths the transport plan. The plot in Fig. 6 shows that for $\epsilon \in [0.5, 2.5]$, the gradient estimates become highly stable (low variance) and align closely with the target gradient (red dashed line).

The right plot in Fig. 6 shows a linear relationship between the number of iterations and execution time. We found that 50 iterations provided a sufficient balance, ensuring that the transport matrix constraints are met without incurring unnecessary computational overhead.

Sinkhorn Algorithm

```

1: Function Potentials( $a, b, u, v$ )
2: Local variables:  $f, g \in \mathbb{R}^N$ 
3: Initialize:  $f = 0, g = 0$ 
4: Set  $C \leftarrow uu^\top + vv^\top - 2uv^\top$ 
5: while stopping criterion not met do
6:   for  $i \in [N]$  do
7:      $f_i \leftarrow \frac{1}{2} (f_i + T_\epsilon(b, g, C_{i:}))$ 
8:      $g_i \leftarrow \frac{1}{2} (g_i + T_\epsilon(a, f, C_{:i}))$ 
9:   end for
10: end while
11: Return  $f, g$ 

```

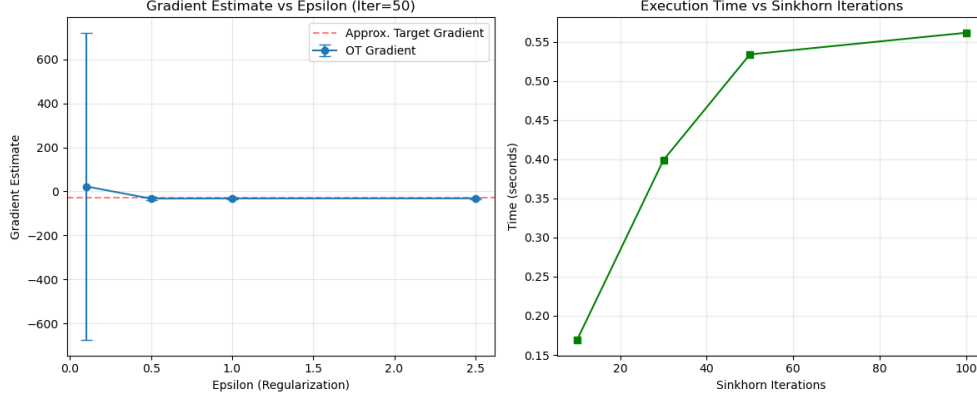


Figure 6: The impact of the regularization parameter ϵ and Sinkhorn iterations on the gradient estimate.

Comment 2.5

ii. There are many other approaches for DPF (see Chen(23)), compare other algorithms(at least two) with OT resampling. Which metrics and diagnostics you can choose? Accuracy? Differentiability? Efficiency?

Response:

Although parameter estimation for particle filters has long been an active research topic, many existing approaches are restricted to settings in which some model parameters are known or the model must satisfy specific structural assumptions. Recently, differentiable particle filters (DPFs) have emerged as a promising alternative, in which particle filter components are parameterized by neural networks and optimized end-to-end via gradient-based learning. For example, Jonschkowski et al. [Jonschkowski et al., 2018] learn an observation likelihood by mapping observations to a feature space and evaluating particle states through a neural network. Karkus et al. [Karkus et al., 2018] apply spatial transformer networks to generate particle-conditioned local maps from a global map and compare them with observations using a neural network. Furthermore, Wen et al. [Wen et al., 2020] extract features from observations and states separately and define the likelihood via the cosine similarity between the resulting embeddings. In addition, Corenflos et al. [Corenflos et al., 2021] propose an entropy-regularised optimal transport-based resampling scheme that renders the resampling step deterministic and differentiable via the Sinkhorn algorithm.

To evaluate the robustness and accuracy of the differentiable particle filter algorithms, we designed a complex experimental scenario based on the “Lorenz 63 Attractor”, a canonical 3D chaotic system. The ground truth state dynamics are governed by the following system of non-linear differential equations:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= x(\rho - z) - y \\ \frac{dz}{dt} &= xy - \beta z\end{aligned}\tag{8}$$

with parameters $\sigma = 10, \rho = 28, \beta = 8/3$, discretized with a time step of $dt = 0.02$.

To simulate a realistic and challenging environment, we introduced “Non-Gaussian noise” to test the filters’ ability to handle outliers and heavy-tailed distributions:

- **Process Noise:** The state evolution is perturbed by Student-t distributed noise ($df = 3$), which has significantly heavier tails than a standard Gaussian, introducing occasional large jumps in the state trajectory.
- **Observation Model:** The observation $y_t \in \mathbb{R}^2$ consists of the x -coordinate and the non-linear Euclidean distance from the origin:

$$y_t = [x_t, \sqrt{x_t^2 + y_t^2 + z_t^2}]^T + \nu_t\tag{9}$$

where the observation noise ν_t follows a Laplace distribution, which is peakier and has heavier tails than a Gaussian.

For the evaluation, we compared the following four baseline configurations on this task using $N = 100$ particles over $T = 200$ time steps.

- Jonschkowski(18) [Jonschkowski et al., 2018]: Using standard hard systematic resampling.
- Wen(21) [Wen et al., 2020]: Using standard hard systematic resampling (identical to Jonschkowski for inference).
- Karkus(18) [Karkus et al., 2018]: Using soft resampling with importance weights ($\alpha = 0.5$).
- Corenflos(21) [Corenflos et al., 2021]: Using differentiable regularized resampling (Sinkhorn entropy regularization).

The primary evaluation metric is the Root Mean Square Error (RMSE) of the estimated mean state trajectory versus the ground truth.

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{x}}_t - \mathbf{x}_t^{\text{true}}\|^2} \quad (10)$$

As illustrated in Figure 7, all particle filters successfully track the complex geometry of the Lorenz attractor. However, the component-wise error analysis in Figure 8 highlights a key divergence in stability: the soft and regularized methods generally maintain smoother error trajectories, contrasting with the abrupt, high-magnitude corrections frequently observed in the hard resampling baselines.

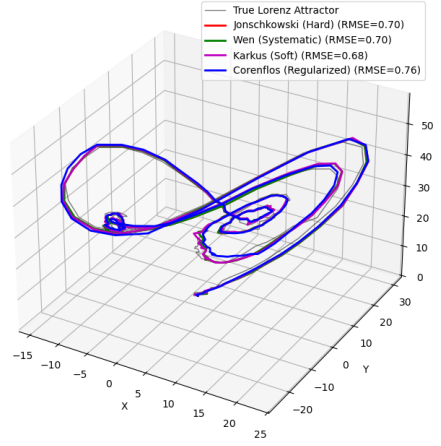


Figure 7: 3D Particle Filter Comparison on Lorenz 63.

This empirical divergence is rooted in the fundamental algorithmic properties summarized in Table 2. The Hard systematic resampling employed by Jonschkowski (2018) and Wen (2021), while computationally efficient, relies on an aggressive culling mechanism. As seen in the error spikes, this approach tends to prematurely discard particles representing abrupt transitions induced by heavy-tailed noise before they can be validated by future observations, resulting in moderate overall performance ($\text{RMSE} \approx 0.70$).

In contrast, Karkus (2018) achieved the lowest error ($\text{RMSE} \approx 0.68$) and notably smoother error profiles. By employing a 'soft' resampling strategy that mixes importance weights with a uniform distribution, this method preserves particle diversity, thereby inherently improving robustness against outliers in the Student- t process noise.

Finally, Corenflos (2021) enables fully differentiable resampling via entropy-regularized optimal transport. However, the requisite regularization imposes a smoothing effect on the transport plan. While this ensures end-to-end differentiability, it manifests empirically as a higher error ($\text{RMSE} \approx 0.76$); the regularization 'over-smooths' the particle distribution, causing a lagged response to the sharp, chaotic dynamics of the attractor."

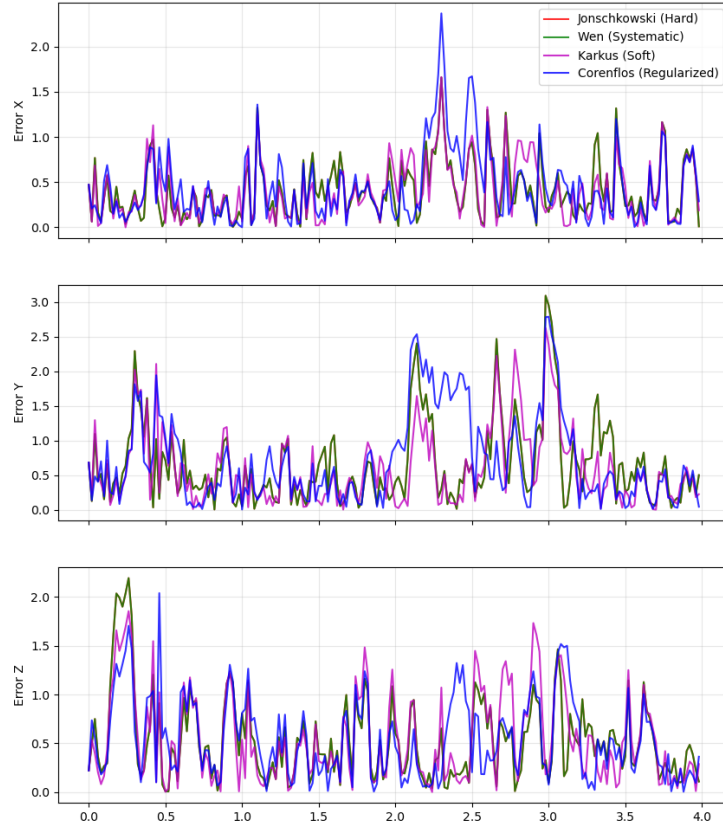


Figure 8: Absolute State Estimation Error over Time (Lorenz 63).

Table 2: RMSE Comparison of Differentiable Particle Filter Methods

Method	Resampling strategy	RMSE ($\downarrow$)
Jonschkowski(18) [Jonschkowski et al., 2018]	Hard Resampling	0.7012
Wen(21) [Wen et al., 2020]	Systematic Resampling	0.7012
Karkus(18) [Karkus et al., 2018]	Soft Resampling	0.6831
Corenflos(21) [Corenflos et al., 2021]	Regularized Resampling	0.7623

References

- [Corenflos et al., 2021] Corenflos, A., Thornton, J., Deligiannidis, G., and Doucet, A. (2021). Differentiable particle filtering via entropy-regularized optimal transport.
- [Daum et al., 2017] Daum, F., Huang, J., and Noushin, A. (2017). Generalized gromov method for stochastic particle flow filters. In *Signal Processing, Sensor/Information Fusion, and Target Recognition XXVI*, volume 10200, page 102000I. SPIE.
- [Jonschkowski et al., 2018] Jonschkowski, R., Rastogi, D., and Brock, O. (2018). Differentiable particle filters: End-to-end learning with algorithmic priors.
- [Karkus et al., 2018] Karkus, P., Hsu, D., and Lee, W. S. (2018). Particle filter networks with application to visual localization.
- [Wen et al., 2020] Wen, H., Chen, X., Papagiannis, G., Hu, C., and Li, Y. (2020). End-to-end semi-supervised learning for differentiable particle filters.